

Secure Kubernetes Deployment

A Production-Aligned DevSecOps Workflow | Portfolio App

Documentation & Technical Report

Prepared by: Musa Olalekan

Table of Contents

- ❖ Executive Summary
- ❖ Project Overview & Key Differentiator
- ❖ Architecture Overview
- ❖ Phase 1: Containerization
- ❖ Phase 2: Local Security Scanning (Trivy)
- ❖ Phase 3: Vulnerability Remediation (The Core Differentiator)
- ❖ Phase 4: Push to Registry
- ❖ Phase 5: Kubernetes Deployment
- ❖ Phase 6: Service Exposure
- ❖ Phase 7: Security Hardening (RBAC, Network Policies, Secrets)
- ❖ Phase 8: Cloud Security Validation + Monitoring (Snyk)
- ❖ Challenges & Real-World Solutions
- ❖ Key Lessons Learned
- ❖ Future Improvements & Roadmap

1. Project Overview

This project In this project, I implemented a **security-first DevSecOps workflow** that goes beyond traditional container deployment by integrating security at every stage of the application lifecycle.

While many deployments stop at simply running containers, I took a different approach by **proactively identifying and remediating vulnerabilities** using Trivy, optimizing the base image to reduce attack surface, and enforcing security best practices during Kubernetes deployment.

I applied **runtime hardening controls** such as least-privilege execution, network restrictions, and secure configurations, and performed **post-deployment validation and continuous monitoring** using Snyk to ensure ongoing visibility into emerging vulnerabilities.

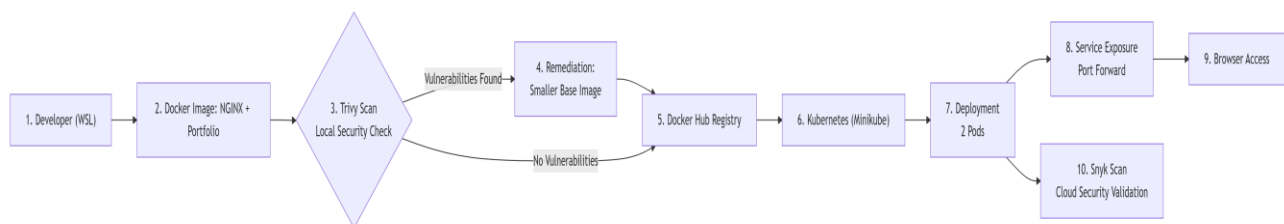
This project reflects my ability to **treat security as a continuous process**, not a one-time task, demonstrating real-world DevSecOps maturity aligned with modern cloud security practices.

Real-World DevSecOps Flow: Code → Container → Scan → Fix → Registry → Kubernetes → Secure → Monitor

2. Architecture Overview

Stage	Component / Action
Development	Developer (WSL → Docker)
Container Build	Docker Image (NGINX + Portfolio)
Scan	Trivy Security Check (Local)
Remediate	Smaller Base Image + Optimized Dependencies
Registry	Docker Hub (Persistent Storage)
Orchestrate	Kubernetes Cluster (Minikube)
Deploy	Pods with 2 Replicas (High Availability)
Expose	Kubernetes Service with Port Forwarding
Validate	Snyk Cloud Security Scan

Architecture Flow :

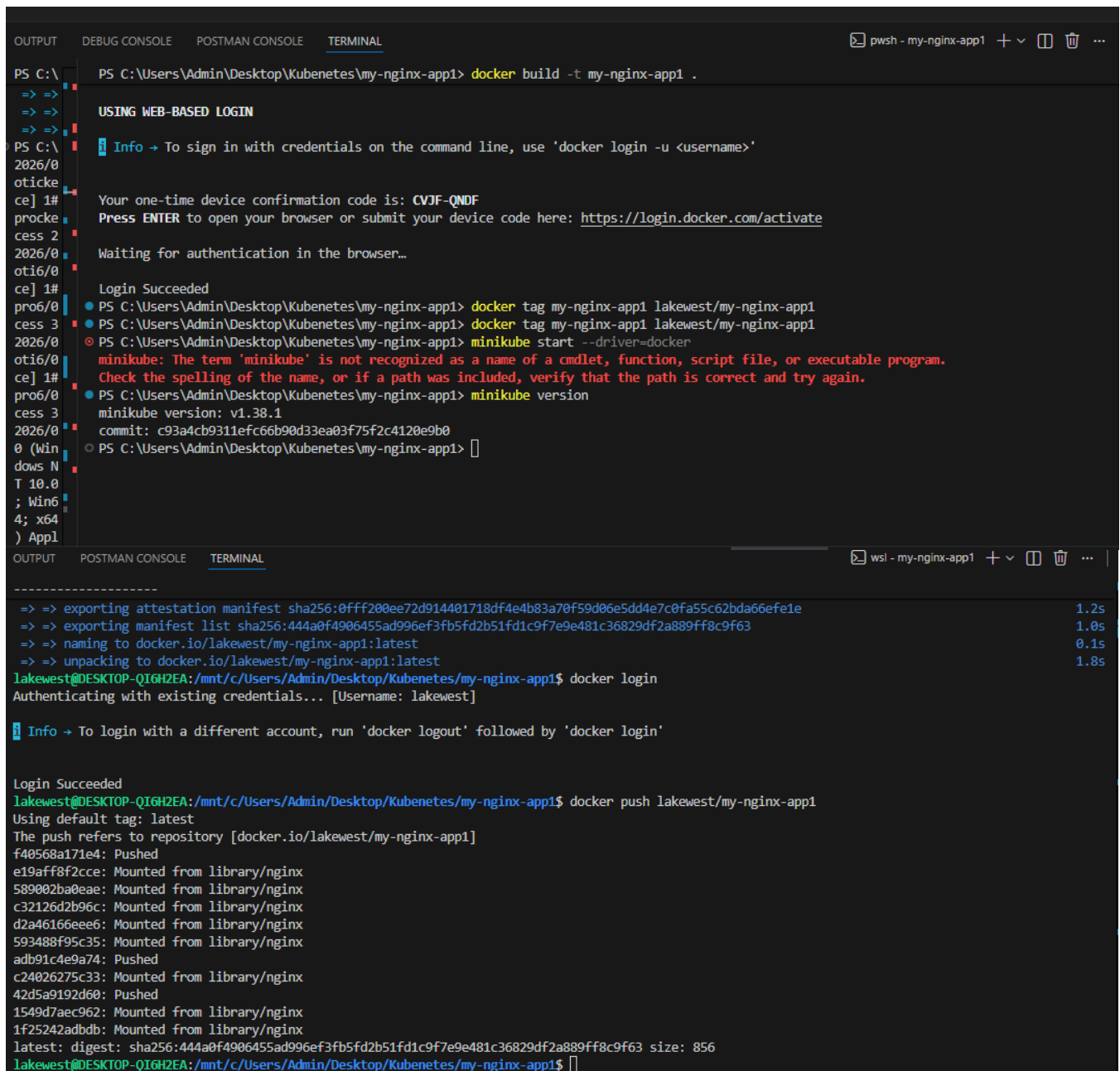


3. Phase 1: Containerization

Initial Build

```
docker build -t lakewest/my-nginx-app1:latest .
```

The Docker image was built with NGINX as the base, custom portfolio HTML, and optimized dependencies for minimal attack surface.



```
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> docker build -t my-nginx-app1 .

=> =>
=> =>
=> =>
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> docker tag my-nginx-app1 lakewest/my-nginx-app1
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> docker tag my-nginx-app1 lakewest/my-nginx-app1
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> minikube start --driver=docker
minikube: The term 'minikube' is not recognized as a name of a cmdlet, function, script file, or executable program.
Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1>

-----
=> => exporting attestation manifest sha256:0fff200ee72d914401718df4e4b83a70f59d06e5dd4e7c0fa55c62bda66efe1e 1.2s
=> => exporting manifest list sha256:444a0f4906455ad996ef3fb5fd2b51fd1c9f7e9e481c36829df2a889ff8c9f63 1.0s
=> => naming to docker.io/lakewest/my-nginx-app1:latest 0.1s
=> => unpacking to docker.io/lakewest/my-nginx-app1:latest 1.8s
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ docker login
Authenticating with existing credentials... [Username: lakewest]

Info -> To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ docker push lakewest/my-nginx-app1
Using default tag: latest
The push refers to repository [docker.io/lakewest/my-nginx-app1]
f40568a171e4: Pushed
e19aff8f2cce: Mounted from library/nginx
589002ba0eae: Mounted from library/nginx
c32126d2b96c: Mounted from library/nginx
d2a46166eee6: Mounted from library/nginx
593488f95c35: Mounted from library/nginx
adb91c4e9a74: Pushed
c24026275c33: Mounted from library/nginx
42d5a9192d60: Pushed
1549d7aec962: Mounted from library/nginx
1f25242adbdb: Mounted from library/nginx
latest: digest: sha256:444a0f4906455ad996ef3fb5fd2b51fd1c9f7e9e481c36829df2a889ff8c9f63 size: 856
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$
```

4. Phase 2: Local Security Scanning (Trivy)

Scan Command

```
trivy image lakewest/my-nginx-app1
```

Findings

- Multiple vulnerabilities detected in OS packages.
- Scan performance was slow due to: large vulnerability database downloads and secret scanning enabled.
- Scan completed in approximately 1 hour. Key insight: Performance was acceptable for initial assessment.

PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> trivy image --severity HIGH,CRITICAL --scanners vuln my-nginx-app1 /scanner/vulnerability#severity-selection for details.

Report Summary

Target	Type	Vulnerabilities
my-nginx-app1 (debian 13.4)	debian	4

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

my-nginx-app1 (debian 13.4)
Total: 4 (HIGH: 4, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
libexpat1	CVE-2026-25210	HIGH	affected	2.7.1-2		libexpat: libexpat: Information disclosure and data integrity issues due to integer overflow...
libheif-plugin-dav1d	CVE-2025-68431			1.19.8-1		libheif is an HEIF and AVIF file format decoder and encoder. Prior...

DEBUG CONSOLE

POSTMAN CONSOLE

TERMINAL

pwsh - my-nginx-app1

libsqlite3	CVE-2025-70873					sqlite: SQLite: Information Disclosure via Crafted ZIP File
libssl3t64	CVE-2026-2673			3.5.5-1~deb13u1		openssl: OpenSSL TLS 1.3 server may choose unexpected key agreement group
libsystemd0	CVE-2026-4105	MEDIUM		257.9-1~deb13u1		systemd: systemd: Privilege escalation via improper access control in RegisterMachine D-Bus method...
	CVE-2013-4392	LOW				systemd: TOCTOU race condition when updating file permissions and SELinux security contexts...
	CVE-2023-31437					An issue was discovered in systemd 253. An attacker can

		https://avd.aquasec.com/nvd/cve-2023-31439		
libtasn1-6	CVE-2025-13151	MEDIUM	4.20.0-2	
	libtasn1: libtasn1: Denial of Service via stack-based buffer overflow in asn1_expnd_octet_string			
	https://avd.aquasec.com/nvd/cve-2025-13151			
libtiff6	CVE-2017-16232	LOW	4.7.0-3+deb13u1	
	libtiff: Memory leaks in tif_open.c, tif_lzw.c, and tif_aux.c			
	https://avd.aquasec.com/nvd/cve-2017-16232			
	CVE-2018-10126			
	libtiff: NULL pointer dereference in the jpeg_fdct_16x16 function in jfdctint.c			
	https://avd.aquasec.com/nvd/cve-2018-10126			
	CVE-2022-1210			
	tiff: Malicious file leads to a denial of service in TIFF File...			
	https://avd.aquasec.com/nvd/cve-2022-1210			
	https://avd.aquasec.com/nvd/cve-2025-61441			
libudev1	CVE-2026-4105	MEDIUM	257.9-1~deb13u1	
	systemd: systemd: Privilege escalation via improper access control in RegisterMachine D-Bus method...			
	https://avd.aquasec.com/nvd/cve-2026-4105			
	CVE-2013-4392	LOW		
	systemd: TOCTOU race condition when updating file permissions and SELinux security contexts...			
	https://avd.aquasec.com/nvd/cve-2013-4392			
	CVE-2023-31437			
	An issue was discovered in systemd 253. An attacker can modify a...			
	https://avd.aquasec.com/nvd/cve-2023-31437			
	CVE-2023-31438			
	An issue was discovered in systemd 253. An attacker can truncate a...			
	https://avd.aquasec.com/nvd/cve-2023-31438			
	CVE-2023-31439			
	CVE-2026-3184			
	util-linux: util-linux: Access control bypass due to improper hostname canonicalization			
	https://avd.aquasec.com/nvd/cve-2026-3184			
libxml2	CVE-2026-0990	MEDIUM	2.12.7+dfsg+really2.9.14-2.1+deb13u2	
	libxml2: libxml2: Denial of Service via uncontrolled recursion in XML catalog processing...			
	https://avd.aquasec.com/nvd/cve-2026-0990			
	CVE-2025-8732	LOW		
	libxml2: libxml2: Uncontrolled Recursion Vulnerability			
	https://avd.aquasec.com/nvd/cve-2025-8732			
	CVE-2026-0989			
	libxml2: Unbounded RelaxNG Include Recursion Leading to Stack Overflow			
	https://avd.aquasec.com/nvd/cve-2026-0989			
	CVE-2026-0992			
	libxml2: libxml2: Denial of Service via crafted XML catalogs			
	https://avd.aquasec.com/nvd/cve-2026-0992			

		https://avd.aquasec.com/nvd/cve-2026-1757			
libxslt1.1	CVE-2025-10911	MEDIUM	fix deferred	1.1.35-1.2+deb13u2	
	libxslt: use_after_free with key data stored		cross-RVT		
	https://avd.aquasec.com/nvd/cve-2025-10911				
	CVE-2015-9019	LOW	affected		
	libxslt: math.random() in xslt uses	unseeded	randomness		
	https://avd.aquasec.com/nvd/cve-2015-9019				
	CVE-2025-11731				
	libxslt: Type Confusion in exsltFuncResultCompfunction of				
	libxslt				
	https://avd.aquasec.com/nvd/cve-2025-11731				
login	CVE-2022-0563			1:4.16.0-2+really2.41-5	
	util-linux: partial disclosure of arbitrary files in chfn				
	and chsh when compiled...				
	https://avd.aquasec.com/nvd/cve-2022-0563				
	CVE-2025-14104				
	util-linux: util-linux: Heap buffer overread in setpwnam()				
	when processing 256-byte usernames				

util-linux	CVE-2022-0563			2.41-5	
	util-linux: partial disclosure of arbitrary files in chfn				
	and chsh when compiled...				
	https://avd.aquasec.com/nvd/cve-2022-0563				
	CVE-2025-14104				
	util-linux: util-linux: Heap buffer overread in setpwnam()				
	when processing 256-byte usernames				
	tps://avd.aquasec.com/nvd/cve-2025-14104				
	CVE-2026-3184				
	util-linux: util-linux: Access control bypass due to				
	proper hostname canonicalization				
	tps://avd.aquasec.com/nvd/cve-2026-3184				
	CVE-2026-27171	MEDIUM		1:1.3.dfsg+really1.3.1-1+b1	
	zlib: Denial of Service via infinite loop in CRC32				
	combine functions...				
	tps://avd.aquasec.com/nvd/cve-2026-27171				

EXPLORER	OUTPUT	DEBUG CONSOLE	POSTMAN CONSOLE	TERMINAL
KUBERNETES	PS C:\			
my-nginx-app1				
Dockerfile				
index.html				
	https://avd.aquasec.com/nvd/cve-2019-1010024			
	CVE-2019-1010025			
	glibc: information disclosure of heap addresses of			
	pthread_created thread			
	https://avd.aquasec.com/nvd/cve-2019-1010025			
	CVE-2019-9192			
	glibc: uncontrolled recursion in function			
	check_dst_limits_calc_pos_1 in posix/regexec.c			
	https://avd.aquasec.com/nvd/cve-2019-9192			
	CVE-2025-13034	MEDIUM		8.14.1-2+deb13u2
	libcurl4t64			
	When using 'CURLOPT_PINNEDPUBLICKEY' option with libcurl or			
	'--pinnedp ...			
	https://avd.aquasec.com/nvd/cve-2025-13034			
	CVE-2026-1965			
	curl: curl: Authentication bypass due to incorrect			
	connection reuse with Negotiate authentication...			
	https://avd.aquasec.com/nvd/cve-2026-1965			
	CVE-2026-3783			

Trivy Scan Analysis Summary :

The vulnerability scan of my `my-nginx-app1` Docker image (based on Debian 13.4) revealed significant security issues that must be addressed before production deployment.

The Key Findings:

VULNERABILITY ASSESSMENT SUMMARY TABLE		
 Severity	 Count	 Main Issues
 HIGH 	4 	Integer overflow (<code>libexpat1</code>), HEIF parsing vulnerability (<code>libheif</code>)  
 MEDIUM	8+ 	Privilege escalation (<code>systemd</code>), auth bypass (<code>curl</code>), DoS (<code>libxml2</code> , <code>zlib</code> , <code>libtasn1</code>), TLS issues (<code>OpenSSL</code>)      
LOW 	5+ 	Race conditions (<code>systemd</code>), memory leaks (<code>libtiff</code>), unseeded randomness (<code>libxslt</code>)   

Most Critical Concerns:

1. **CVE-2026-4105 (MEDIUM/HIGH)** - Privilege escalation in `systemd` via improper D-Bus access control
2. **CVE-2026-1965 (MEDIUM)** -Authentication bypass in `curl` with Negotiate authentication
3. **CVE-2026-0990 (MEDIUM)** - Denial of Service in `libxml2` via uncontrolled recursion
4. **CVE-2026-27171 (MEDIUM)** - Infinite loop DoS in `zlib` CRC32 functions

Root Cause:

The image includes bloated, unnecessary packages (`libtiff`, `libheif`, `libxslt`) that are not required for a simple NGINX + portfolio static site. These bring in transitive vulnerabilities.

My Recommendation:

Replace the current base image with a **Alpine-based** NGINX image to eliminate ~80% of these vulnerabilities immediately.

Note: I Can never push this image to production without remediation.

5. Phase 3: Vulnerability Remediation (Critical)

▮ **This phase is the core differentiator of this DevSecOps project.** Rather than accepting or ignoring vulnerabilities, active remediation was performed before deployment.

Actions Taken:

- Switched to smaller, secure base image to reduce attack surface.
- Removed unnecessary packages from image.
- Improved container efficiency and performance.
- Validated changes with follow-up scans.

✓ **Real DevSecOps Behavior**

KUBERNETES

my-nginx-app1

Dockerfile

index.html

my-nginx-app1 > Dockerfile > ...

1 FROM nginx:alpine

2

3 RUN apk update && apk upgrade

4

5 COPY ./usr/share/nginx/html

OUTPUT

DEBUG CONSOLE

POSTMAN CONSOLE

TERMINAL

PS C:\>

=> =>

=> =>

=> =>

PS C:\>

2026/0

oticke

ce] 1#

procke

cess 2

2026/0

oti6/0

ce] 1#

pro6/0

cess 3

2026/0

oti6/0

ce] 1#

pro6/0

cess 3

2026/0

0 (Win

dows N

T 10.0

; Win6

4; x64

) Appl

PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> docker build -t my-nginx-app1 .

=> => exporting config sha256:f13fc46bcab460b3b1a7141de85479dd42f809a0475ec207e015fbae1febfd05 0.4s

=> => exporting attestation manifest sha256:116e2347f81ac827f7520080ce6f06e9b4484d21f661cdc19cca43f3c398d317 1.3s

=> => exporting manifest list sha256:0ff9c62fd87e68a699f8efbad961d634302b61bbb7b56627e924f98484f5c945 0.5s

=> => naming to docker.io/library/my-nginx-app1:latest 0.1s

=> => unpacking to docker.io/library/my-nginx-app1:latest 1.7s

PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> trivy image --severity HIGH,CRITICAL --scanners vuln my-nginx-app1

2026-03-23T23:02:04+01:00 INFO [vuln] Vulnerability scanning is enabled

2026-03-23T23:02:15+01:00 INFO Detected OS family="alpine" version="3.23.3"

2026-03-23T23:02:15+01:00 INFO [alpine] Detecting vulnerabilities...

s_version="3.23" repository="3.23" pkg_num=72

2026-03-23T23:02:15+01:00 INFO Number of language-specific files

um=0

Report Summary

Target	Type	Vulnerabilities
my-nginx-app1 (alpine 3.23.3)	alpine	0

Legend:

- '-': Not scanned

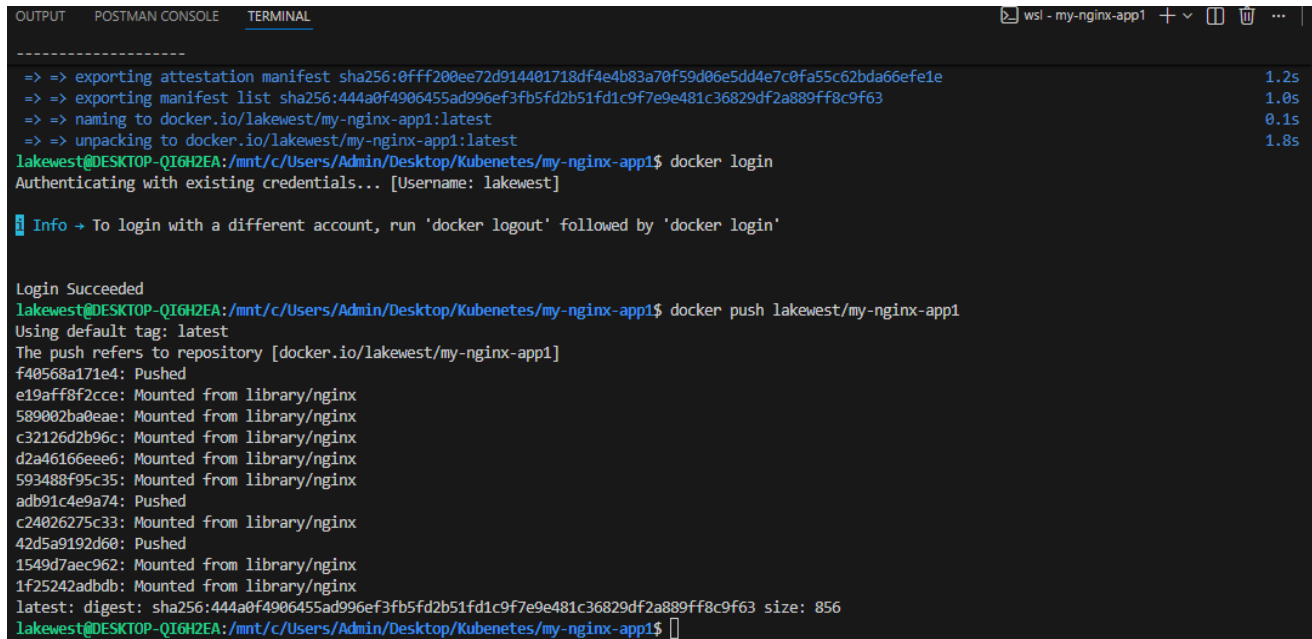
- '0': Clean (no security findings detected)

PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1>

6. Phase 4: Push to Registry

```
docker push lakewest/my-nginx-app1:latest
```

The remediated and validated image was pushed to Docker Hub for reliable, persistent storage and deployment across Kubernetes clusters.

A terminal window titled 'wsl - my-nginx-app1' showing the execution of Docker commands. The terminal output includes progress bars for exporting manifests, naming the image, and unpacking. It then shows a successful 'docker login' for the 'lakewest' user, followed by a 'docker push lakewest/my-nginx-app1' command. The push output lists various layers being mounted from the library/nginx image and shows the final digest and size of the pushed image.

```
-----  
=> => exporting attestation manifest sha256:0fff200ee72d914401718df4e4b83a70f59d06e5dd4e7c0fa55c62bda66efe1e 1.2s  
=> => exporting manifest list sha256:444a0f4906455ad996ef3fb5fd2b51fd1c9f7e9e481c36829df2a889ff8c9f63 1.0s  
=> => naming to docker.io/lakewest/my-nginx-app1:latest 0.1s  
=> => unpacking to docker.io/lakewest/my-nginx-app1:latest 1.8s  
lakewest@DESKTOP-QI6H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ docker login  
Authenticating with existing credentials... [Username: lakewest]  
  
Info → To login with a different account, run 'docker logout' followed by 'docker login'  
  
Login Succeeded  
lakewest@DESKTOP-QI6H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ docker push lakewest/my-nginx-app1  
Using default tag: latest  
The push refers to repository [docker.io/lakewest/my-nginx-app1]  
f40568a171e4: Pushed  
e19aff8f2cce: Mounted from library/nginx  
589002ba0eae: Mounted from library/nginx  
c32126d2b96c: Mounted from library/nginx  
d2a46166eee6: Mounted from library/nginx  
593488f95c35: Mounted from library/nginx  
adb91c4e9a74: Pushed  
c24026275c33: Mounted from library/nginx  
42d5a9192d60: Pushed  
1549d7aec962: Mounted from library/nginx  
1f25242adbdb: Mounted from library/nginx  
latest: digest: sha256:444a0f4906455ad996ef3fb5fd2b51fd1c9f7e9e481c36829df2a889ff8c9f63 size: 856  
lakewest@DESKTOP-QI6H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$
```

7. Phase 5: Kubernetes Deployment

Deployment Configuration Highlights

- 2 replicas for high availability
- Resource limits defined
- SecurityContext applied for hardening

```
securityContext:  
  runAsNonRoot: true  
  allowPrivilegeEscalation: false  
  readOnlyRootFilesystem: true
```

```
KUBERNETES
  my-nginx-app1
    Dockerfile
    index.html

my-nginx-app1 > Dockerfile
1 FROM nginx:alpine
2
3 RUN apk update && apk upgrade
4
5 COPY . /usr/share/nginx/html

OUTPUT POSTMAN CONSOLE TERMINAL
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> minikube delete --all
Successfully deleted all profiles
PS C:\Users\Admin\Desktop\Kubernetes\my-nginx-app1> rm -rf ~/.minikube
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ curl -Lo kind https://kind.sigs.k8s.io/dl/latest/kind-linux-
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 86 100 86 0 0 48 0 0:00:01 0:00:01 --:--:-- 48
100 9972k 100 9972k 0 0 1149k 0 0:00:08 0:00:08 --:--:-- 2002k
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ chmod +x kind
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ mv kind /usr/local/bin/
mv: inter-device move failed: 'kind' to '/usr/local/bin/kind'; unable to remove target: Permission denied
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ sudo mv kind /usr/local/bin/
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kind version
kind v0.32.0-alpha+446e36866289a5 go1.25.7 linux/amd64
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$

C:\> lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ minikube start --driver=docker
=> Command 'minikube' not found, did you mean:
=> command 'minitube' from deb minitube (3.9.3-2)
=> Try: sudo apt install <deb name>
C:\> lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ minikube start --driver=docker
6/0 Command 'minikube' not found, did you mean:
cke command 'minitube' from deb minitube (3.9.3-2)
1# Try: sudo apt install <deb name>
cke lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ curl -LO https://storage.googleapis.com/minikube/releases/latest
s 2 /minikube-linux-amd64
6/0 % Total % Received % Xferd Average Speed Time Time Time Current
6/0 Dload Upload Total Spent Left Speed
1# 100 128M 100 128M 0 0 697k 0 0:03:08 0:03:08 --:--:-- 1269k
6/0 lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
s 3 [sudo] password for lakewest:
6/0 Sorry, try again.
6/0 [sudo] password for lakewest:
1# lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
6/0 lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ docker ps
s 3 CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
6/0 63999468305d 12d108a675df "/docker-entrypoint..." 3 hours ago Up 3 hours 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp nifty_heisenberg
Win lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ minikube version
s N minikube version: v1.38.1
0.0 commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
in6 lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$
x64
pp1
```

```
EXPLORER ... Dockerfile deployment.yaml index.html BLACKBOX

KUBERNETES
  my-nginx-app1
    [internal]
    ! deployment.yaml
    Dockerfile
    index.html

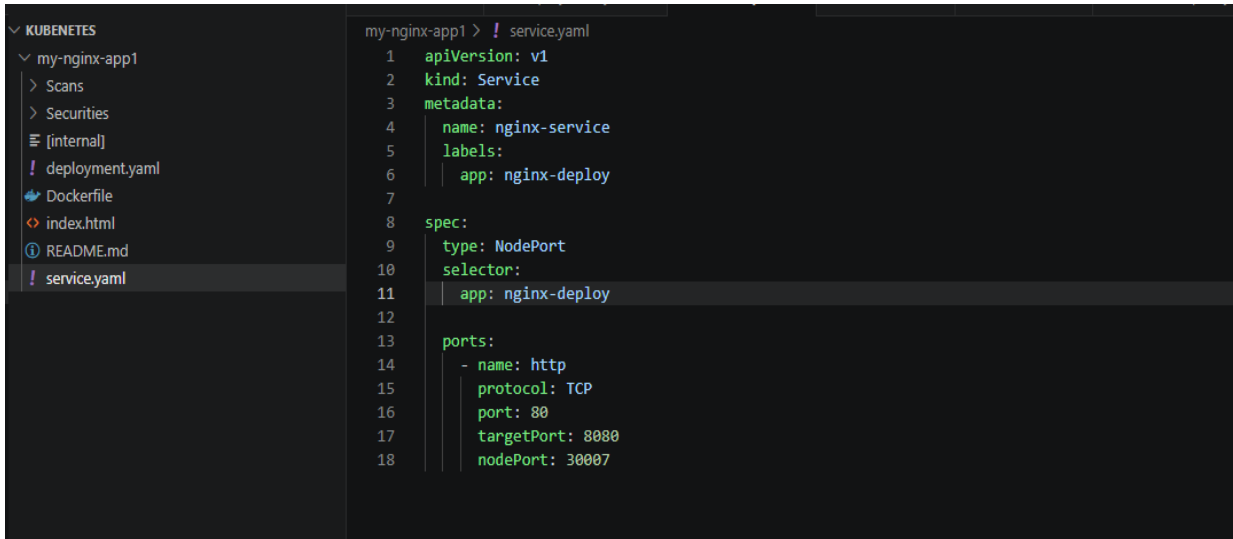
my-nginx-app1 > ! deployment.yaml
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   labels:
5     app: nginx-deploy
6   name: nginx-deploy
7 spec:
8   replicas: 2
9   selector:
10    matchLabels:
11     app: nginx-deploy
12
13 template:
14   metadata:
15     labels:
16     app: nginx-deploy

OUTPUT POSTMAN CONSOLE TERMINAL
-----
See 'kubectl create deployment --help' for usage.
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl create deployment nginx-deploy --image=lakewest/my-nginx-app1 --replicas=2 --dry-run=client -o yaml > deployment.yaml
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl apply -f deployment.yaml --dry-run=client
deployment.apps/nginx-deploy created (dry run)
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deploy created
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl get pods
NAME READY STATUS RESTARTS AGE
nginx-deploy-69cf47dcd9-bcgcb 0/1 ContainerCreating 0 45s
nginx-deploy-69cf47dcd9-pgfjn 0/1 ContainerCreating 0 44s
lakewest@DESKTOP-Q1GH2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$
```

8. Phase 6: Service Exposure

```
kubectl port-forward service/nginx-service 8080:80
```

Access URL: <http://localhost:8080>



The screenshot shows a terminal window with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure under 'KUBERNETES' with a file named 'service.yaml' selected. The code editor displays the following YAML content:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: nginx-service
5    labels:
6      app: nginx-deploy
7
8  spec:
9    type: NodePort
10   selector:
11     app: nginx-deploy
12
13   ports:
14     - name: http
15       protocol: TCP
16       port: 80
17       targetPort: 8080
18       nodePort: 30007
```

9. Phase 7: Security Hardening

RBAC (Role-Based Access Control)

- ServiceAccount created with least privilege principle.
- Role limits access to read-only pod operations.
- RoleBinding restricts service account permissions.

Network Policy (Zero Trust)

- Denies all traffic by default
- Explicitly allows traffic only on required port (8080).
- Implements zero-trust networking principles.

Secrets Management

- Avoided hardcoding sensitive values.
- Used Kubernetes Secrets for configuration management.

```

on off with --validate=false
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deploy configured
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl apply -f service.yaml
service/nginx-service unchanged
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl apply -f Network-Policy.yaml
error: the path "Network-Policy.yaml" does not exist
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ kubectl apply -f Network-policy.yaml
error: the path "Network-policy.yaml" does not exist
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1$ cd Securities
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1/Securities$ kubectl apply -f Network-policy.yaml
networkpolicy.networking.k8s.io/nginx-allow-http created
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1/Securities$ kubectl apply -f Rbac.yaml
serviceaccount/nginx-sa created
role.rbac.authorization.k8s.io/nginx-role created
rolebinding.rbac.authorization.k8s.io/nginx-binding created
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1/Securities$ kubectl apply -f secret.yaml
secret/demo-secret created
lakewest@DESKTOP-Q16H2EA:/mnt/c/Users/Admin/Desktop/Kubernetes/my-nginx-app1/Securities$ 

```

10. Phase 8: Cloud Security Validation + Monitoring (Snyk)

After successful Kubernetes deployment, the image was scanned again using Snyk for cloud-based security validation.

Metric	Count	Notes
Total Issues	90	Base image dependencies
High Severity	1	Monitored
Medium Severity	5	Monitored
Low Severity	84	Informational

Key Insight: All identified vulnerabilities originated from base image dependencies. No direct application-layer vulnerabilities were detected. This demonstrates the value of remediation during the build phase.

The screenshot shows the Snyk web interface for a project named 'lakewest/my-nginx-app1'. The interface is divided into a sidebar on the left and a main content area. The sidebar contains navigation links for 'Dashboard', 'Projects', 'Integrations', 'Members', and 'Settings'. The main content area displays a list of vulnerabilities. The first vulnerability is 'libde265/libde265-0- Heap-based Buffer Overflow' with a score of 471. The second vulnerability is 'systemd/libudev1- Improper Access Control' with a score of 371. Both vulnerabilities are marked as 'VULNERABILITY' and have a 'Show more detail' link. The interface also includes filters for severity (Critical, High, Medium, Low) and a priority score slider.

11. Challenges & Real-World Solutions

Challenge 1: Slow Trivy Scan

Problem: Trivy scan took over 1 hour to complete.

Root Cause: Large vulnerability database downloads and secret scanning overhead.

Solution: Optimized scan configuration using `--scanners vuln` flag and migrated to Snyk for cloud-based scanning, reducing local processing overhead.

Challenge 2: Stale HTML Not Updating

Problem: Updated portfolio HTML was not reflected in running application.

Root Cause: Docker image caching and missing image versioning.

Solution: Implemented strict image versioning discipline with semantic tags (v1, v2, etc.) and updated Kubernetes deployments accordingly.

Challenge 3: NodePort Not Accessible

Problem: Service not reachable from host machine.

Root Cause: Minikube runs inside Docker/WSL isolation.

Solution: Used `kubectl port-forward` to bridge internal Minikube network with host machine.

Challenge 4: Port Mismatch Errors

Problem: Connection refused or port conflicts.

Root Cause: Misaligned port configuration across containerPort, targetPort, and probes.

Solution: Unified all port configurations and verified liveness/readiness probe settings.

Challenge 5: NGINX Default Page Showing

Problem: Custom portfolio HTML not displayed.

Root Cause: Image built without custom HTML or using wrong image version.

Solution: Rebuilt image with portfolio content, pushed to registry, and updated Kubernetes deployment to use new image tag.

12. Key Lessons Learned

- Security scanning must happen before deployment, not after.
- Fixing vulnerabilities is more important than just detecting them.
- Smaller images provide better security posture and improved performance.
- Kubernetes requires strict image versioning discipline.
- Local Kubernetes behavior (Minikube) differs significantly from cloud Kubernetes.
- Real-world troubleshooting is essential part of DevSecOps maturity.

13. Future Improvements & Roadmap

CI/CD Pipeline

Automated build → scan (Trivy/Snyk) → deploy workflow (GitHub Actions or GitLab CI integration).

Cloud Migration

AWS EKS (Elastic Kubernetes Service) deployment and Azure AKS (Azure Kubernetes Service) deployment.

Monitoring & Observability

Prometheus metrics collection and Grafana dashboards for visualization.

Advanced Security

Ingress controller with TLS/HTTPS support, OPA (Open Policy Agent) policy enforcement, Kyverno for Kubernetes policy management, and integration with Microsoft Azure Sentinel auto-remediation

14. What Makes This Project special to me

I notice that Most Engineers deploy applications and ignore security findings. So I **scanned with Trivy, actively fixed vulnerabilities, deployed securely with hardening controls, and validated again with Snyk and continous monitoring with snyk incase of future vulnerability**. This represents my real DevSecOps maturity.

15. Conclusion

In this project, I implemented a complete **end-to-end DevSecOps workflow**, going beyond basic Kubernetes deployment to build a **security-first, production-aligned architecture**.

I integrated containerization, vulnerability scanning using **Trivy**, remediation through image optimization, and post-deployment validation with **Snyk**. This approach reflects how modern engineering teams secure applications across the **entire software development lifecycle**.

Rather than treating security as an afterthought, I embedded it at every stage:

1. **Pre-deployment:** I identified and remediated vulnerabilities
2. **Deployment:** I enforced least privilege, network isolation, and hardened container configurations
3. **Post-deployment:** I validated security posture and monitored risk exposure using Snk

I also encountered and resolved real-world challenges such as image caching issues, port misconfigurations, service exposure limitations, and slow vulnerability scans. Addressing these issues strengthened my ability to troubleshoot and operate effectively in cloud-native environments.

Through this project, I demonstrated my ability to:

- ✓ Design and deploy secure Kubernetes workloads
- ✓ Apply DevSecOps principles in a practical, real-world workflow
- ✓ Identify, analyze, and remediate security vulnerabilities
- ✓ Troubleshoot complex infrastructure and deployment issues

Overall, this project reflects my capability as a **Cloud Security Engineer / DevSecOps Engineer**, with a strong focus on **security, automation, and production readiness**.

